Wazuh + OpenCTI Bidirectional Threat Intelligence Integration

Complete Production Writeup — AegisryxLabs

Author: Md. Abu Saeid

Role: SOC Analyst | Incident Responder | SIEM Specialist

Lab: AegisryxLabs

Completed: June 2026

Status: ✅ Fully Operational in Lab Environment

Table of Contents

1. Project Overview

This project builds a **fully operational, bidirectional, real-time threat intelligence pipeline** between Wazuh (SIEM/XDR) and OpenCTI (Cyber Threat Intelligence Platform).

Every log event processed by Wazuh is automatically checked against OpenCTI's live threat intelligence database via GraphQL API. When a match is found, Wazuh fires an enriched alert, creates a STIX sighting in OpenCTI, and delivers a real-time notification to Telegram — all within seconds, with zero manual intervention.

The result:

- No CDB list files on disk
- No sync delays
- Real-time GraphQL queries on every alert
- Full bidirectional feedback loop
- 8+ sightings created automatically in OpenCTI

2. Why This Project

Problems with Traditional SIEM Operations

Problem	Impact
Alert fatigue	SOC analysts miss real threats
Intelligence silos	IOC data never reaches detection tools
Manual correlation	Hours wasted on VirusTotal lookups
Static threat lists	Miss newly discovered indicators
No feedback loop	Detection events never enrich the CTI platform

Two Approaches Implemented

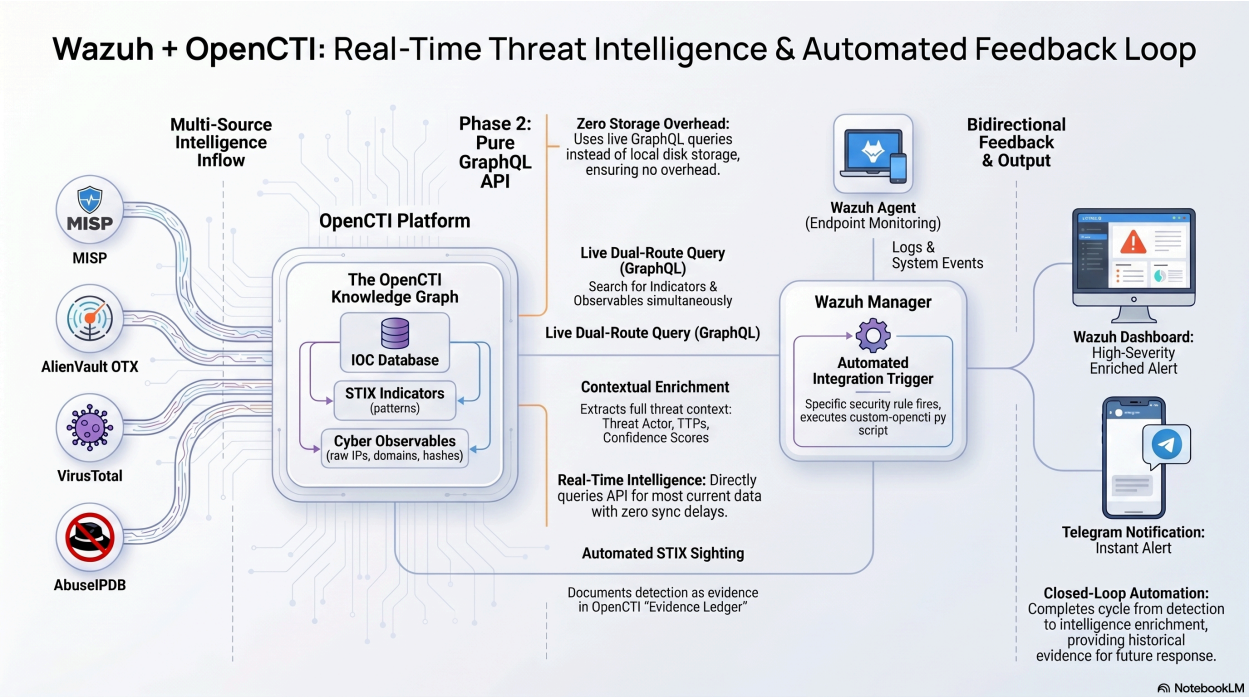
Phase 1 — CDB List Approach

- Pull IOCs from OpenCTI every 6 hours via cron
- Store them in Wazuh CDB list files
- Wazuh matches logs against files
- Problem: storage grows, 6h gap misses new IOCs

Phase 2 — Pure GraphQL API (Ram Kumar's Vision)

- No files on disk
- Every alert triggers a live GraphQL query to OpenCTI
- Searches both Indicators AND Observables
- Real-time, unlimited IOC coverage
- This is the production-ready approach

3. Architecture



```
External Threat Feeds
(MISP, AlienVault OTX, AbuseIPDB,
 MalwareBazaar, VirusTotal, abuse.ch)
              ↓ ingestion connectors
         OpenCTI Platform
         192.168.0.104:8080
         ├── IOC Database
         ├── Indicators (STIX patterns)
         ├── Observables (raw IPs/domains)
         └── Sightings (when/where seen)
              ↓ GraphQL API (real-time)
    Wazuh Manager (KVM Ubuntu Server)
    192.168.122.51
    ├── custom-opencti.py (integration)
    ├── Detection rules (100100–100302)
    ├── Wazuh socket (/var/ossec/queue/sockets/queue)
    └── analysisd → enriched alerts
              ↓ alerts
    Wazuh Agents (monitored endpoints)
    ├── ubuntu1 (192.168.122.51)
    └── ubuntu2 (192.168.122.63)
              ↓ rule 100302 fires
    ┌──────────────────────────────┐
    │ Wazuh Dashboard (OpenSearch)│ ✅
    │ OpenCTI Sighting created    │ ✅
    │ Telegram alert delivered    │ ✅
    └──────────────────────────────┘
```

4. Environment Setup

Lab Infrastructure

Component	Details
OpenCTI host	Ubuntu Desktop, Docker Compose
OpenCTI IP	192.168.0.104:8080
Wazuh Manager	Ubuntu 22.04 Server, KVM VM
Wazuh Manager IP	192.168.122.51
Wazuh Agent	Ubuntu 22.04 VM
Wazuh Agent IP	192.168.122.63
OpenSearch/Indexer	192.168.122.183:9200
Python version	3.10 (Wazuh internal), 3.x (system)

OpenCTI Connectors Configured

- AlienVault OTX
- AbuseIPDB
- MalwareBazaar
- VirusTotal
- MITRE ATT&CK
- Abuse.ch

5. OpenCTI Deployment (Docker)

OpenCTI was deployed using its official Docker repository to act as the central Threat Intelligence Platform (TIP). The environment was configured by setting up the `.env` file with administrative credentials, the base URL, and a generated UUID for API authentication. After spinning up the containers via `docker-compose`, the OpenCTI web interface was accessed to retrieve the API token required for future integrations.
(Note: For the detailed, step-by-step installation and configuration commands, please refer to my dedicated OpenCTI lab writeup.)

6. Wazuh Deployment (Manager & Agent)

Wazuh was installed to provide comprehensive security monitoring and log analysis. Using the official Wazuh APT repository, both the Wazuh Manager and Agent were installed on the target infrastructure. The agent was configured to communicate with the manager, and both services were started and enabled to run on boot.
(Note: The complete installation and configuration steps are documented in my separate Wazuh lab writeup.)

7. Python Dependencies for Integration

To enable seamless communication between Wazuh and OpenCTI, the necessary Python libraries (`pycti` and `requests`) were installed in two distinct environments:

- **System Python:** Used for the standalone IoC synchronization scripts.

- **Wazuh Internal Python:** Located at `/var/ossec/framework/python/`, used for Wazuh's custom integration scripts.

Verifying the installation in both environments ensured that both the external sync tools and Wazuh's internal active response mechanisms could successfully interact with the OpenCTI API.

8. Part 4 — CDB List Integration (Phase 1)

> **Note:** This is the initial approach. Phase 2 (GraphQL) replaces this for production. Keep Phase 1 as a backup/reference.

8a. IOC Sync Script

bash

```
sudo nano /usr/local/bin/opencti-wazuh-sync.py
```

Key configuration in script:

python

```
OPENCTI_URL   = "http://192.168.0.104:8080"
OPENCTI_TOKEN = "your-token-here"
WAZUH_CDB_PATH = "/var/ossec/etc/lists"
IOC_MAX_AGE_DAYS = 180
MIN_CONFIDENCE = 50
```

Run initial sync:

bash

```
sudo chmod +x /usr/local/bin/opencti-wazuh-sync.py
sudo /usr/local/bin/opencti-wazuh-sync.py
```

Expected output:

```
✓ ipv4: 38 IOCs → /var/ossec/etc/lists/opencti-ipv4
✓ domain: 176 IOCs → /var/ossec/etc/lists/opencti-domain
✓ url: 42 IOCs → /var/ossec/etc/lists/opencti-url
✓ md5: 124 IOCs → /var/ossec/etc/lists/opencti-md5
✓ sha1: 57 IOCs → /var/ossec/etc/lists/opencti-sha1
✓ sha256: 191 IOCs → /var/ossec/etc/lists/opencti-sha256
✓ Total IOCs: 628
```

Set up cron (every 6 hours):

bash

```
sudo crontab -e
# Add:
0 */6 * * * /usr/local/bin/opencti-wazuh-sync.py >> /var/log/opencti-sync.log 2>&1
```

9. Part 5 — Pure GraphQL Integration (Phase 2)

This is the **production-ready approach** that replaces CDB lists entirely.

9a. Shell Wrapper

bash

```
sudo nano /var/ossec/integrations/custom-opencti
```

9a. Shell Wrapper

bash

```
sudo nano /var/ossec/integrations/custom-opencti
```

sh

```
#!/bin/sh
WPYTHON_BIN="framework/python/bin/python3"
SCRIPT_PATH_NAME="$0"
DIR_NAME="$(cd $(dirname ${SCRIPT_PATH_NAME}); pwd -P)"
SCRIPT_NAME="$(basename ${SCRIPT_PATH_NAME})"

case ${DIR_NAME} in
    */integrations)
        if [ -z "${WAZUH_PATH}" ]; then
            WAZUH_PATH="$(cd ${DIR_NAME}/..; pwd)"
        fi
        PYTHON_SCRIPT="${DIR_NAME}/${SCRIPT_NAME}.py"
    ;;
esac

${WAZUH_PATH}/${WPYTHON_BIN} ${PYTHON_SCRIPT} "$@"
```

bash

```
sudo chmod 750 /var/ossec/integrations/custom-opencti
sudo chown root:wazuh /var/ossec/integrations/custom-opencti
```

9b. Python Integration Script

Key configuration constants:

python

```
OPENCTI_URL       = "http://192.168.0.104:8080"
OPENCTI_TOKEN     = "your-token-here"
LOG_FILE          = "/var/ossec/logs/opencti-integration.log"
MIN_SCORE         = 30
WAZUH_SOCKET      = "/var/ossec/queue/sockets/queue"
WAZUH_IDENTITY_ID = "cfd35842-a11d-46a9-be76-71854ee088fe"
```

> **IMPORTANT:** `WAZUH_IDENTITY_ID` is the OpenCTI System identity ID for "Wazuh SIEM". Get it from: OpenCTI → Settings → Accesses → or by running `get_or_create_identity("Wazuh SIEM")` once manually.

How the script works:

```
Alert received from Wazuh
        ↓
Extract observables:
  srcip, dstip → IPv4
  domain, hostname → Domain
  url, http_url → URL
  md5/sha1/sha256 → Hash
  filename → File
  win.eventdata.* → Process
        ↓
For each observable:
  Step 1: search_indicator(STIX pattern)
    → If found + score >= MIN_SCORE:
      → create_sighting(indicator_id, WAZUH_IDENTITY_ID)
      → send enriched alert to Wazuh socket
  Step 2: search_observable(value) [fallback]
    → If found:
      → get linked indicator ID if available
      → create_sighting(linked_id OR observable_id)
      → send enriched alert to Wazuh socket
```

CRITICAL BUGS FIXED (see Part 11):

- `count` → must be `attribute_count` in sighting mutation
- `objectLabel` → flat list `[{value: "tag"}]` not `edges/node`
- GraphQL variable `$value` broken for `stixCyberObservables` → use inline f-string
- `identityAdd` mutation → must use inline f-string not variables
- Script output (stdout) → must go to Wazuh socket, not stdout

bash

custom-opencti.py

bash

```
sudo nano /var/ossec/integrations/custom-opencti.py
```

```
#!/usr/bin/env python3
"""
Wazuh-OpenCTI Pure GraphQL Integration
======================================
- No CDB lists needed
- Queries OpenCTI Indicators AND Observables directly
- Supports: IPv4, IPv6, Domain, Hostname, URL,
            Filename, MD5, SHA1, SHA256, Windows process
- Creates STIX sightings automatically
- Sends enriched alerts via Wazuh socket
- Zero file storage overhead

Place in : /var/ossec/integrations/custom-opencti.py
Ownership: root:wazuh
Perms    : 750
"""

import sys
import json
import logging
import ssl
import socket
import urllib.request
from datetime import datetime, timezone

# ── CONFIGURATION ──────────────────────────────────────────
OPENCTI_URL       = "http://192.168.0.104:8080"   # <-- Chang
e me
OPENCTI_TOKEN     = "flgrn_octi_tkn_3fJ8ncDqT8CsWXJhp5jiJjWwK
4lQCfmyArhU_eM21NiG831dykEFskjC6zVtcHoj"  # <-- Change me
LOG_FILE          = "/var/ossec/logs/opencti-integration.log"
```

```
MIN_SCORE         = 30   # minimum IOC score to alert on
WAZUH_SOCKET      = "/var/ossec/queue/sockets/queue"
WAZUH_IDENTITY_ID = "cfd35842-a11d-46a9-be76-71854ee088fe"  #
Wazuh SIEM identity in OpenCTI
# ──────────────────────────────────────────────────────────────

logging.basicConfig(
    filename=LOG_FILE,
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s",
)

SSL_CTX = ssl.create_default_context()
SSL_CTX.check_hostname = False
SSL_CTX.verify_mode    = ssl.CERT_NONE


# ═══════════════════════════════════════════════════════════════
# OpenCTI GraphQL Client
# ═══════════════════════════════════════════════════════════════

class OpenCTIGraphQL:
    """Direct GraphQL API client — no pycti dependency"""

    def __init__(self):
        self.url = f"{OPENCTI_URL}/graphql"
        self.h   = {
            "Authorization": f"Bearer {OPENCTI_TOKEN}",
            "Content-Type":  "application/json",
        }

    def query(self, q, v=None):
        """Execute a GraphQL query/mutation"""
        try:
            payload = json.dumps({
                "query": q, "variables": v or {}
```

```
            }).encode()
            req = urllib.request.Request(
                self.url, data=payload,
                headers=self.h, method="POST"
            )
            with urllib.request.urlopen(
                req, timeout=10, context=SSL_CTX
            ) as r:
                return json.loads(r.read())
        except Exception as e:
            logging.error(f"GraphQL failed: {e}")
            return None

    # ── Search Indicator ──────────────────────────────

    def search_indicator(self, pattern):
        """Search Indicator by STIX pattern (inline value)"""
        safe = (pattern.replace("\\", "\\\\")
                       .replace('"', '\\"'))
        r = self.query(f"""
        query {{
          indicators(
            filters:{{
              mode:and
              filters:[{{key:"pattern",values:["{safe}"],oper
ator:eq}}]
              filterGroups:[]
            }}
            first:1
          ) {{
            edges{{node{{
              id name pattern x_opencti_score
              objectLabel{{value}}
              createdBy{{...on Identity{{name}}}}
              killChainPhases{{
                edges{{node{{kill_chain_name phase_name}}}}
```

```
              }}
            }}}}
          }}
        }}""")
        if not r:
            return None
        e = r.get("data",{}).get("indicators",{}).get("edge
s",[])
        return e[0]["node"] if e else None

    # ── Search Observable ──────────────────────────────

    def search_observable(self, value):
        """Search Observable by value (inline value)"""
        safe = value.replace('"', '\\"')
        r = self.query(f"""
        query {{
          stixCyberObservables(
            filters:{{
              mode:and
              filters:[{{key:"value",values:["{safe}"],operat
or:eq}}]
              filterGroups:[]
            }}
            first:1
          ) {{
            edges{{node{{
              id entity_type
              ...on IPv4Addr   {{value}}
              ...on IPv6Addr   {{value}}
              ...on DomainName {{value}}
              ...on Url        {{value}}
              ...on Hostname   {{value}}
              objectLabel{{value}}
              createdBy{{...on Identity{{name}}}}
              indicators{{
```

```
                edges{{node{{id name x_opencti_score}}}}
              }}
            }}}}
          }}
        }}""")
        if not r:
            return None
        e = r.get("data",{}).get("stixCyberObservables",{}).g
et("edges",[])
        return e[0]["node"] if e else None

    # ── Get or Create Identity ──────────────────────────

    def get_or_create_identity(self, name, desc=""):
        """Get existing or create new STIX Identity"""
        r = self.query("""
        query FindIdentity($name: String!) {
          identities(
            filters:{
              mode:and
              filters:[{key:"name",values:[$name],operator:e
q}]
              filterGroups:[]
            }
            first:1
          ) { edges{node{id name}} }
        }""", {"name": name})
        if r:
            e = r.get("data",{}).get("identities",{}).get("ed
ges",[])
            if e:
                return e[0]["node"]["id"]

        # Create with inline value
        sn = name.replace('"', '\\"')
        sd = desc.replace('"', '\\"')
```

```
        r2 = self.query(f"""
        mutation {{
          identityAdd(input:{{
            type:"System"
            name:"{sn}"
            description:"{sd}"
          }}) {{id}}
        }}""")
        if r2:
            return r2.get("data",{}).get("identityAdd",{}).ge
t("id")
        return None

    # ── Create Sighting ──────────────────────────────────

    def create_sighting(self, from_id, to_id, desc, ts):
        """
        Create STIX sighting relationship.
        Field is attribute_count (NOT count).
        """
        r = self.query("""
        mutation CreateSighting(
          $fromId:StixRef! $toId:StixRef!
          $description:String!
          $first_seen:DateTime! $last_seen:DateTime!
        ) {
          stixSightingRelationshipAdd(input:{
            fromId:$fromId toId:$toId
            description:$description
            first_seen:$first_seen last_seen:$last_seen
            attribute_count:1 confidence:75
          }){id}
        }""", {
            "fromId":      from_id,
            "toId":        to_id,
            "description": desc,
```

```
            "first_seen":  ts,
            "last_seen":   ts,
        })
        if r and r.get("data",{}).get("stixSightingRelationsh
ipAdd"):
            sid = r["data"]["stixSightingRelationshipAdd"]["i
d"]
            logging.info(f"Sighting created: {sid}")
            return sid
        errs = (r or {}).get("errors", [])
        logging.warning(f"Sighting failed: {errs}")
        return None


# ═══════════════════════════════════════════════════════════
# Observable Extractor
# ═══════════════════════════════════════════════════════════

class ObservableExtractor:
    """Extract all observable types from a Wazuh alert"""

    def extract(self, alert):
        obs  = []
        data = alert.get("data", {})

        # IPv4
        for f in ["srcip","dstip","src_ip","dst_ip","remote_i
p"]:
            v = data.get(f)
            if v and self._pub(v):
                obs.append({"type":"ipv4","value":v,
                    "pattern":f"[ipv4-addr:value = '{v}']"})

        # IPv6
        for f in ["srcip6","dstip6","src_ipv6"]:
            v = data.get(f)
```

```
            if v and ":" in v:
                obs.append({"type":"ipv6","value":v,
                    "pattern":f"[ipv6-addr:value = '{v}']"})

        # Domain / Hostname
        for f in ["domain","hostname","dns_query",
                  "http_hostname","dest_hostname","zeek.dns.q
uery"]:
            v = data.get(f)
            if v and self._dom(v):
                obs.append({"type":"domain","value":v,
                    "pattern":f"[domain-name:value =
'{v}']"})

        # URL
        for f in ["url","http_url","request_url",
                  "full_url","zeek.http.uri"]:
            v = data.get(f)
            if v:
                obs.append({"type":"url","value":v,
                    "pattern":f"[url:value = '{v}']"})

        # File hashes
        for hf, sk in [
            ("md5","MD5"),("sha1","SHA-1"),("sha256","SHA-25
6"),
            ("md5_after","MD5"),("sha1_after","SHA-1"),
            ("sha256_after","SHA-256"),
        ]:
            v = data.get(hf)
            if v and len(v) in (32,40,64):
                obs.append({"type":f"hash_{hf}","value":v,
                    "pattern":f"[file:hashes.'{sk}' =
'{v}']"})

        # Filenames
```

```
        for f in ["filename","file_name","syscheck.path"]:
            v = data.get(f)
            if v:
                obs.append({"type":"filename","value":v,
                    "pattern":f"[file:name = '{v}']"})

        # Windows process
        for f in ["win.eventdata.image",
                  "win.eventdata.commandLine",
                  "win.eventdata.parentImage"]:
            v = data.get(f)
            if v:
                obs.append({"type":"process","value":v,
                    "pattern":f"[process:command_line =
'{v}']"})

        return obs

    def _pub(self, ip):
        """Return True only for public routable IPs"""
        try:
            p = ip.split(".")
            if len(p) != 4:
                return False
            if not all(0 <= int(x) <= 255 for x in p):
                return False
            if ip.startswith(("10.","127.","169.254.","0.","2
55.")):
                return False
            if ip.startswith("192.168."):
                return False
            if ip.startswith("172.") and 16 <= int(p[1]) <= 3
1:
                return False
            return True
        except Exception:
```

```
            return False

    def _dom(self, d):
        """Basic domain validation"""
        if not d or len(d) < 4 or "." not in d:
            return False
        if d.startswith(("127.","localhost")):
            return False
        return True


# ═══════════════════════════════════════════════════════════
# Main Integration Class
# ═══════════════════════════════════════════════════════════

class WazuhOpenCTIIntegration:

    def __init__(self):
        self.api = OpenCTIGraphQL()
        self.ext = ObservableExtractor()
        logging.info("Integration initialized")

    # ── Send to Wazuh socket ──────────────────────────────

    def _socket(self, enriched):
        """Send enriched alert to Wazuh analysisd via socke
t"""
        try:
            msg = "1:custom-opencti:" + json.dumps(enriched)
            with socket.socket(
                socket.AF_UNIX, socket.SOCK_DGRAM
            ) as s:
                s.connect(WAZUH_SOCKET)
                s.send(msg.encode())
            logging.info(
                f"Sent to socket: {enriched.get('ioc_valu
```

```
e')}"
            )
        except Exception as e:
            logging.error(f"Socket failed: {e}")
            print(json.dumps(enriched))  # fallback stdout

    # ── Parse labels ──────────────────────────────────────

    def _labels(self, raw):
        """Handle flat list or edges/node label formats"""
        if isinstance(raw, list):
            return [
                l.get("value","") for l in raw
                if isinstance(l, dict)
            ]
        return [
            e["node"]["value"]
            for e in raw.get("edges",[])
        ]

    # ── Create sighting (cached identity) ─────────────────

    def _sighting(self, from_id, agent_name, agent_ip,
                  rule_id, rule_desc, ts):
        """
        Create sighting using cached WAZUH_IDENTITY_ID.
        Falls back to dynamic identity lookup if needed.
        """
        # Use cached Wazuh SIEM identity — guaranteed to work
        agent_id = WAZUH_IDENTITY_ID

        # Fallback: try to find/create agent-specific identit
y
        if not agent_id:
            agent_id = self.api.get_or_create_identity(
                agent_name, f"Wazuh Agent at {agent_ip}"
```

```
            )
        if not agent_id:
            logging.warning(
                f"Cannot get identity for {agent_name}"
            )
            return

        desc = (
            f"Detected by Wazuh on agent {agent_name} "
            f"({agent_ip}). Rule {rule_id}: {rule_desc}"
        )
        self.api.create_sighting(from_id, agent_id, desc, ts)

    # ── Build enriched alert dict ──────────────────────────

    def _enriched(self, obs, ind_id, name, score, labels,
                  phases, actor, mtype,
                  rule_id, rule_desc, agent_name, agent_ip):
        return {
            "opencti_match":    True,
            "match_type":       mtype,
            "ioc_type":         obs["type"],
            "ioc_value":        obs["value"],
            "indicator_id":     ind_id,
            "indicator_name":   name,
            "indicator_score":  score,
            "indicator_labels": labels,
            "kill_chain":       phases,
            "threat_actor":     actor,
            "original_alert": {
                "rule_id":    rule_id,
                "rule_desc":  rule_desc,
                "agent_name": agent_name,
                "agent_ip":   agent_ip,
            },
        }
```

```
    # ── Process one observable ──────────────────────────

    def _process(self, obs, agent_name, agent_ip,
                 rule_id, rule_desc, ts):

        # ── Step 1: Search Indicator ────────────────────
        ind = self.api.search_indicator(obs["pattern"])

        if ind:
            score = ind.get("x_opencti_score", 0) or 0
            if score < MIN_SCORE:
                logging.debug(
                    f"Score {score} below minimum {MIN_SCOR
E}"
                )
                return

            labels = self._labels(ind.get("objectLabel",[]))
            cb     = ind.get("createdBy", {}) or {}
            actor  = cb.get("name", "Unknown")
            phases = [
                e["node"]["phase_name"]
                for e in ind.get(
                    "killChainPhases",{}
                ).get("edges",[])
            ]

            logging.info(
                f"INDICATOR MATCH: {obs['type']} = "
                f"{obs['value']} (score:{score})"
            )

            self._sighting(
                ind["id"], agent_name, agent_ip,
                rule_id, rule_desc, ts
```

```
            )
            self._socket(self._enriched(
                obs, ind["id"],
                ind.get("name", obs["value"]),
                score, labels, phases, actor,
                "indicator",
                rule_id, rule_desc, agent_name, agent_ip
            ))
            logging.info(
                f"Enriched alert (indicator): {obs['value']}"
            )
            return

        # ── Step 2: Fallback to Observable ──────────────
        if obs["type"] not in ("ipv4","ipv6","domain","url"):
            return

        logging.info(
            f"No indicator — checking observable: {obs['valu
e']}"
        )
        ob = self.api.search_observable(obs["value"])

        if not ob:
            logging.debug(f"No match: {obs['value']}")
            return

        labels = self._labels(ob.get("objectLabel",[]))
        cb     = ob.get("createdBy", {}) or {}
        actor  = cb.get("name", "Unknown")
        ie     = ob.get("indicators",{}).get("edges",[])
        score  = (
            ie[0]["node"].get("x_opencti_score",50) or 50
        ) if ie else 50
        ind_id = ie[0]["node"].get("id") if ie else None
```

```
            logging.info(
                f"OBSERVABLE MATCH: {obs['type']} = "
                f"{obs['value']} (author:{actor})"
            )

            # Prefer linked indicator for sighting
            from_id = ind_id if ind_id else ob["id"]
            label   = "linked-indicator" if ind_id else "observab
le-direct"
            logging.info(
                f"Creating sighting via {label}: {from_id}"
            )
            self._sighting(
                from_id, agent_name, agent_ip,
                rule_id, rule_desc, ts
            )

            self._socket(self._enriched(
                obs, ob["id"], obs["value"],
                score, labels, [], actor,
                "observable",
                rule_id, rule_desc, agent_name, agent_ip
            ))
            logging.info(
                f"Enriched alert (observable): {obs['value']}"
            )

    # ── Main alert processor ──────────────────────────────

    def process_alert(self, alert):
            rule_id    = alert.get("rule",{}).get("id","?")
            rule_desc  = alert.get("rule",{}).get("descriptio
n","Alert")
            agent_name = alert.get("agent",{}).get("name","Unknow
n")
            agent_ip   = alert.get("agent",{}).get("ip","Unknow
```

```
n")

        logging.info(f"Processing alert: {rule_id}")

        observables = self.ext.extract(alert)
        if not observables:
            logging.debug("No observables extracted")
            return

        raw_ts = alert.get("timestamp")
        try:
            dt = datetime.fromisoformat(raw_ts)
            if not dt.tzinfo:
                dt = dt.replace(tzinfo=timezone.utc)
            dt = dt.astimezone(timezone.utc)
        except Exception:
            dt = datetime.now(timezone.utc)
        ts = (
            dt.isoformat(timespec="milliseconds")
              .replace("+00:00","Z")
        )

        for obs in observables:
            logging.info(
                f"Checking {obs['type']}: {obs['value']}"
            )
            try:
                self._process(
                    obs, agent_name, agent_ip,
                    rule_id, rule_desc, ts
                )
            except Exception as e:
                logging.error(
                    f"Error processing {obs['value']}: {e}"
                )
```

```
# ═══════════════════════════════════════════════════════════
# Entry Point
# ═══════════════════════════════════════════════════════════

def main():
    try:
        with open(sys.argv[1]) as f:
            alert = json.load(f)
        WazuhOpenCTIIntegration().process_alert(alert)
    except Exception as e:
        logging.error(f"Fatal error: {e}")


if __name__ == "__main__":
    main()
```

```
sudo chmod 750 /var/ossec/integrations/custom-opencti.py
sudo chown root:wazuh /var/ossec/integrations/custom-opencti.py
```

10. Part 6 — Wazuh Rules

opencti_rules.xml

bash

```
sudo nano /var/ossec/etc/rules/opencti_rules.xml
```

```
<group name="opencti,threat_intel">

  <!-- Base rule: any OpenCTI match -->
  <rule id="100100" level="12">
    <decoded_as>json</decoded_as>
    <field name="opencti_match">\.+</field>
    <description>OpenCTI IOC match: $(ioc_type) = $(ioc_valu
e)</description>
    <group>opencti,threat_detection,</group>
  </rule>
```

```
  <!-- Indicator match -->
  <rule id="100101" level="14">
    <if_sid>100100</if_sid>
    <field name="match_type">^indicator$</field>
    <description>OpenCTI Indicator: $(ioc_value) – $(threat_a
ctor)</description>
    <group>opencti,indicator_match,</group>
  </rule>

  <!-- Observable match -->
  <rule id="100102" level="12">
    <if_sid>100100</if_sid>
    <field name="match_type">^observable$</field>
    <description>OpenCTI Observable: $(ioc_value) – $(threat_
actor)</description>
    <group>opencti,observable_match,</group>
  </rule>

  <!-- Malicious IPv4 -->
  <rule id="100104" level="12">
    <if_sid>100100</if_sid>
    <field name="ioc_type">^ipv4$</field>
    <description>Malicious IP via OpenCTI: $(ioc_value) – $(t
hreat_actor)</description>
    <group>opencti,malicious_ip,</group>
  </rule>

  <!-- Malicious domain -->
  <rule id="100105" level="12">
    <if_sid>100100</if_sid>
    <field name="ioc_type">^domain$</field>
    <description>Malicious domain via OpenCTI: $(ioc_value)</
description>
    <group>opencti,malicious_domain,</group>
  </rule>
```

```
  <!-- Malicious hash -->
  <rule id="100106" level="15">
    <if_sid>100100</if_sid>
    <field name="ioc_type">^hash</field>
    <description>Malicious file hash via OpenCTI: $(ioc_valu
e)</description>
    <group>opencti,malware,</group>
  </rule>

  <!-- Malicious URL -->
  <rule id="100107" level="13">
    <if_sid>100100</if_sid>
    <field name="ioc_type">^url$</field>
    <description>Malicious URL via OpenCTI: $(ioc_value)</des
cription>
    <group>opencti,malicious_url,</group>
  </rule>

  <!-- High score 80+ -->
  <rule id="100108" level="15">
    <if_sid>100100</if_sid>
    <field name="indicator_score" type="pcre2">^([8-9][0-9]|1
00)$</field>
    <description>CRITICAL OpenCTI IOC score 80+: $(ioc_value)
</description>
    <group>opencti,critical_threat,</group>
  </rule>

</group>
```

opencti_enrichment_rules.xml

bash

```
sudo nano /var/ossec/etc/rules/opencti_enrichment_rules.xml
```

```
<group name="opencti,enrichment">

  <!-- Updated enrichment rules — delegates to opencti_rules.
xml -->
  <rule id="100300" level="12">
    <decoded_as>json</decoded_as>
    <field name="opencti_match">\.+</field>
    <description>OpenCTI IOC match: $(ioc_type) = $(ioc_valu
e) — $(threat_actor)</description>
    <group>opencti_enriched,</group>
  </rule>

  <rule id="100301" level="15">
    <if_sid>100300</if_sid>
    <field name="indicator_score" type="pcre2">^([8-9][0-9]|1
00)$</field>
    <description>CRITICAL OpenCTI IOC score 80+: $(ioc_value)
</description>
    <group>critical,double_confirmed,</group>
  </rule>

  <rule id="100302" level="12">
    <if_sid>100300</if_sid>
    <field name="match_type">^observable$</field>
    <description>OpenCTI Observable match: $(ioc_value) — Aut
hor: $(threat_actor)</description>
    <group>opencti,observable_match,</group>
  </rule>

  <rule id="100303" level="14">
    <if_sid>100300</if_sid>
    <field name="match_type">^indicator$</field>
    <description>OpenCTI Indicator match: $(ioc_value) — Acto
r: $(threat_actor)</description>
    <group>opencti,indicator_match,</group>
```

```
  </rule>

  <rule id="100304" level="12">
    <if_sid>100300</if_sid>
    <field name="ioc_type">^ipv4$</field>
    <description>Malicious IP via OpenCTI: $(ioc_value) — $(t
hreat_actor)</description>
    <group>opencti,malicious_ip,</group>
  </rule>

  <rule id="100305" level="12">
    <if_sid>100300</if_sid>
    <field name="ioc_type">^domain$</field>
    <description>Malicious domain via OpenCTI: $(ioc_value)</
description>
    <group>opencti,malicious_domain,</group>
  </rule>

  <rule id="100306" level="15">
    <if_sid>100300</if_sid>
    <field name="ioc_type">^hash</field>
    <description>Malicious file hash via OpenCTI: $(ioc_valu
e)</description>
    <group>opencti,malware,</group>
  </rule>

  <rule id="100307" level="13">
    <if_sid>100300</if_sid>
    <field name="ioc_type">^url$</field>
    <description>Malicious URL via OpenCTI: $(ioc_value)</des
cription>
    <group>opencti,malicious_url,</group>
  </rule>

</group>
```

Set permissions:

bash

```
sudo chown root:wazuh /var/ossec/etc/rules/opencti_*.xml
sudo chmod 660 /var/ossec/etc/rules/opencti_*.xml
```

Test rules with logtest:

bash

```
echo '{"opencti_match": true, "match_type": "observable", "ioc_type": "ipv4", "ioc_value": "47.77.229.36", "indicator_score": 50, "threat_actor": "AbuseIPDB"}' | \
  /var/ossec/bin/wazuh-logtest
```

Expected: Rule 100302 fires.

---

11. Part 7 — ossec.conf Configuration

bash

```
sudo nano /var/ossec/etc/ossec.conf
```

Add inside the file (integration block first):

xml

```
<ossec_config>
  <integration>
    <name>custom-opencti</name>
    <hook_url>http://192.168.0.104:8080</hook_url>
    <level>3</level>
    <alert_format>json</alert_format>
  </integration>
</ossec_config>

<ossec_config>
  <ruleset>
    <list>etc/lists/opencti-ipv4</list>
    <list>etc/lists/opencti-ipv6</list>
    <list>etc/lists/opencti-domain</list>
    <list>etc/lists/opencti-url</list>
    <list>etc/lists/opencti-md5</list>
```

```
    <list>etc/lists/opencti-sha1</list>
    <list>etc/lists/opencti-sha256</list>
    <rule_dir>rules</rule_dir>
  </ruleset>
</ossec_config>
```

> **Important:** `<level>3</level>` ensures SSH failed login (rule 5760, level 5) triggers the integration. Do not set higher than 5.

---

12. Part 8 — Telegram Alert Integration

Create Bot via BotFather

```
1. Telegram → search @BotFather
2. /newbot
3. Name: AegisryxLabs Alerts
4. Username: aegisryxlabs_wazuh_bot
5. Copy the TOKEN
```

Verify Token

bash

```
TOKEN="YOUR_TOKEN_HERE"
curl -s "https://api.telegram.org/bot${TOKEN}/getMe" | python3 -m json.tool
```

Get Channel Chat ID

bash

```
curl -s -X POST \
  "https://api.telegram.org/bot${TOKEN}/sendMessage" \
  -d "chat_id=@your_channel_username" \
  -d "text=Test" | python3 -m json.tool
```

Note the `chat.id` value (negative number starting with `-100`).

Shell Wrapper

bash

```
sudo nano /var/ossec/integrations/custom-telegram
```

Same shell wrapper pattern as custom-opencti (just change filename reference).

Python Script Key Config

python

```
TELEGRAM_TOKEN   = "YOUR_BOT_TOKEN"
TELEGRAM_CHAT_ID = "-1003905043292"
MIN_ALERT_LEVEL  = 5
```

SSL fix for self-signed certificates:

python

```
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE
urllib.request.urlopen(req, timeout=10, context=ctx)
```

Add to ossec.conf

xml

```
<ossec_config>
  <integration>
    <name>custom-telegram</name>
    <hook_url>https://api.telegram.org/botYOUR_TOKEN</hook_ur
l>
    <level>5</level>
    <alert_format>json</alert_format>
  </integration>
</ossec_config>
```

13. Part 9 — IOC Sync Script (CDB Reference)

Location: `/usr/local/bin/opencti-wazuh-sync.py`

Key settings:

python

```
OPENCTI_URL      = "http://192.168.0.104:8080"
OPENCTI_TOKEN    = "your-token"
WAZUH_CDB_PATH   = "/var/ossec/etc/lists"
```

```
WAZUH_RULES_PATH = "/var/ossec/etc/rules"
IOC_MAX_AGE_DAYS = 180
MIN_CONFIDENCE   = 50
```

Cron (every 6 hours):

```
0 */6 * * * /usr/local/bin/opencti-wazuh-sync.py >> /var/log/opencti-sync.log 2>&1
```

---

14. Part 10 — Testing & Validation

Test Detection Pipeline

bash

```
# Step 1: Check IOC list has entries
head -5 /var/ossec/etc/lists/opencti-ipv4

# Step 2: Simulate malicious SSH login
logger -p auth.info -t sshd \
  'Failed password for root from 93.125.114.57 port 22 ssh2'

# Step 3: Watch integration log
tail -f /var/ossec/logs/opencti-integration.log

# Step 4: Watch Wazuh alerts
tail -f /var/ossec/logs/alerts/alerts.log | \
  grep -i "opencti\|100302\|malicious"
```

Expected Log Sequence (Success)

```
INFO - Integration initialized
INFO - Processing alert: 5760
INFO - Checking ipv4: 93.125.114.57
INFO - No indicator — checking observable: 93.125.114.57
INFO - OBSERVABLE MATCH: ipv4 = 93.125.114.57 (author: AlienVault)
INFO - Creating sighting via linked-indicator: xxxxxxxx-xxxx
INFO - Sighting created: yyyyyyyy-yyyy-yyyy-yyyy-yyyyyyyyyyyy
INFO - Sent to socket: 93.125.114.57
INFO - Enriched alert (observable): 93.125.114.57
```

Verify in Wazuh Dashboard

Navigate to: `https://192.168.122.183` → Threat Hunting → Events

Filter: `manager.name: wazuh1`

Expected alert:

```
Rule 100302 — Level 12
OpenCTI Observable match: 93.125.114.57 — Author: AlienVault
```

Verify in OpenCTI

Navigate to: `http://192.168.0.104:8080` → Events → Sightings

Remove "false positive" filter. You should see new sighting:

```
94.58.66.53 → Sighted in/at → Wazuh SIEM
Qualification: True Positive
Confidence: 2 - Probably True
```

Verify Telegram

Check your Wazuh-Alert channel — alert should arrive within 30 seconds.

---

15. Part 11 — Key Bugs Fixed

These are critical issues encountered and resolved during the project. **Memorize these for production deployment.**

Bug 1 — `count` vs `attribute_count` in Sighting Mutation

graphql

```
# WRONG — causes GRAPHQL_VALIDATION_FAILED
stixSightingRelationshipAdd(input:{
  count: 1        ← WRONG FIELD NAME
})

# CORRECT
stixSightingRelationshipAdd(input:{
  attribute_count: 1    ← CORRECT
})
```

Bug 2 — `objectLabel` Structure

python

```
# WRONG — assumes edges/node structure
labels = [e["node"]["value"] for e in
          indicator.get("objectLabel",{}).get("edges",[])]

# CORRECT — flat list in this OpenCTI version
raw = indicator.get("objectLabel", [])
if isinstance(raw, list):
    labels = [l.get("value","") for l in raw if isinstance(l,dict)]
```

Bug 3 — GraphQL Variables Broken for stixCyberObservables

python

```
# WRONG — $value variable not resolved correctly
gql = """
query SearchObservable($value: String!) {
  stixCyberObservables(filters:{
    filters:[{key:"value", values:[$value]}]  ← variable broken
  })
}"""
result = self.query(gql, {"value": ip})

# CORRECT — use inline f-string
safe = value.replace('"', '\\"')
gql = f"""
query {{
  stixCyberObservables(filters:{{
    filters:[{{key:"value",values:["{safe}"]}}]
  }})
}}"""
result = self.query(gql)  # no variables
```

Bug 4 — identityAdd Mutation Variables Broken

python

```
# WRONG
gql = """mutation CreateIdentity($name: String!) {
  identityAdd(input:{name:$name}) {id}
}"""
result = self.query(gql, {"name": name})  # returns None

# CORRECT — inline f-string
sn = name.replace('"', '\\"')
gql = f"""mutation {{
  identityAdd(input:{{type:"System",name:"{sn}"}}) {{id}}
}}"""
result = self.query(gql)  # no variables
```

Bug 5 — Integration Output Goes to stdout, Not Wazuh

python

```
# WRONG — integratord discards stdout
print(json.dumps(enriched))

# CORRECT — send directly to Wazuh socket
msg = "1:custom-opencti:" + json.dumps(enriched)
with socket.socket(socket.AF_UNIX, socket.SOCK_DGRAM) as s:
    s.connect("/var/ossec/queue/sockets/queue")
    s.send(msg.encode())
```

Bug 6 — SSL Certificate Error on Telegram

python

```
# WRONG — fails with self-signed cert chain
urllib.request.urlopen(req, timeout=10)

# CORRECT
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE
urllib.request.urlopen(req, timeout=10, context=ctx)
```

Bug 7 — Identity Returns None → Sighting Skipped

python

```
# WRONG — dynamic lookup sometimes fails
agent_id = self.api.get_or_create_identity(agent_name, ...)
# agent_id can be None → sighting silently skipped

# CORRECT — use cached constant
WAZUH_IDENTITY_ID = "cfd35842-a11d-46a9-be76-71854ee088fe"
agent_id = WAZUH_IDENTITY_ID  # guaranteed to work
```

Bug 8 — ossec.conf Integration Level Too High

xml

```
<!-- WRONG — SSH rule 5760 is level 5, never triggers -->
<level>7</level>

<!-- CORRECT — catches level 3 and above -->
<level>3</level>
```

---

16. Final File Reference

File Locations

File	Location	Purpose
Shell wrapper	`/var/ossec/integrations/custom-opencti`	Calls Python script
Integration script	`/var/ossec/integrations/custom-opencti.py`	GraphQL queries, sightings

File	Location	Purpose
Telegram wrapper	`/var/ossec/integrations/custom-telegram`	Calls Telegram Python
Telegram script	`/var/ossec/integrations/custom-telegram.py`	Sends Telegram alerts
Sync script	`/usr/local/bin/opencti-wazuh-sync.py`	CDB list sync (Phase 1)
Detection rules	`/var/ossec/etc/rules/opencti_rules.xml`	Rules 100100–100108
Enrichment rules	`/var/ossec/etc/rules/opencti_enrichment_rules.xml`	Rules 100300–100307
Main config	`/var/ossec/etc/ossec.conf`	Integration + ruleset config
CDB lists	`/var/ossec/etc/lists/opencti-*`	IOC lookup files
Integration log	`/var/ossec/logs/opencti-integration.log`	Debug and match logs
Telegram log	`/var/ossec/logs/telegram-integration.log`	Telegram send logs
Wazuh alerts	`/var/ossec/logs/alerts/alerts.json`	All alert output

File Permissions

bash

```
# Integration scripts
sudo chmod 750 /var/ossec/integrations/custom-opencti
sudo chmod 750 /var/ossec/integrations/custom-opencti.py
sudo chmod 750 /var/ossec/integrations/custom-telegram
sudo chmod 750 /var/ossec/integrations/custom-telegram.py
sudo chown root:wazuh /var/ossec/integrations/custom-opencti
sudo chown root:wazuh /var/ossec/integrations/custom-opencti.py
sudo chown root:wazuh /var/ossec/integrations/custom-telegram
sudo chown root:wazuh /var/ossec/integrations/custom-telegram.py

# Sync script
sudo chmod +x /usr/local/bin/opencti-wazuh-sync.py

# Rules
sudo chown root:wazuh /var/ossec/etc/rules/opencti_*.xml
sudo chmod 660 /var/ossec/etc/rules/opencti_*.xml
```

17. Production Deployment Checklist

Use this checklist when deploying to any new environment.

Pre-Deployment

- [ ] OpenCTI instance running and accessible
- [ ] OpenCTI API token generated with write permissions
- [ ] Wazuh manager installed and running
- [ ] Network connectivity: Wazuh manager can reach OpenCTI URL
- [ ] Python 3.6+ on Wazuh manager
- [ ] `pycti` installed for system Python AND Wazuh Python

OpenCTI Setup

- [ ] At least one threat intel connector configured (AlienVault, AbuseIPDB, etc.)
- [ ] Connectors are ingesting IOCs (check Observations → Observables)
- [ ] Note down API token
- [ ] Create "Wazuh SIEM" System identity manually OR note the auto-created ID
- [ ] Note down `WAZUH_IDENTITY_ID` from OpenCTI Settings

Wazuh Files

- [ ] `/var/ossec/integrations/custom-opencti` created and permissions set
- [ ] `/var/ossec/integrations/custom-opencti.py` created and permissions set
- [ ] `OPENCTI_URL` correct in script
- [ ] `OPENCTI_TOKEN` correct in script
- [ ] `WAZUH_IDENTITY_ID` correct in script (get from OpenCTI!)
- [ ] `/var/ossec/etc/rules/opencti_rules.xml` created
- [ ] `/var/ossec/etc/rules/opencti_enrichment_rules.xml` created
- [ ] `/var/ossec/etc/ossec.conf` has integration block with `<level>3</level>`

- [ ] `/var/ossec/etc/ossec.conf` has ruleset block with CDB lists

Telegram (Optional)

- [ ] Bot created via @BotFather
- [ ] Token verified with `/getMe`
- [ ] Bot added as admin to channel
- [ ] Chat ID retrieved
- [ ] `custom-telegram.py` configured with token and chat ID
- [ ] SSL context configured (for self-signed certs)
- [ ] Integration block added to ossec.conf

Validation

- [ ] `wazuh-logtest` confirms rule 100302 fires
- [ ] `sudo systemctl restart wazuh-manager` succeeds
- [ ] Test logger command fires alert in `alerts.log`
- [ ] Integration log shows `OBSERVABLE MATCH` or `INDICATOR MATCH`
- [ ] Integration log shows `Sighting created: {uuid}`
- [ ] Wazuh dashboard shows rule 100302 alert
- [ ] OpenCTI Sightings shows new entry (remove false positive filter)
- [ ] Telegram channel receives alert

Key Values to Note for Each Deployment

```
OPENCTI_URL       = http://<ip>:8080
OPENCTI_TOKEN     = <from OpenCTI Profile → API Access>
WAZUH_IDENTITY_ID = <from OpenCTI after first run>
TELEGRAM_TOKEN    = <from @BotFather>
TELEGRAM_CHAT_ID  = <negative number from sendMessage response>
```

Final Summary

This project delivers a complete, production-grade SOC automation pipeline:

```
Threat feed → OpenCTI → GraphQL API → Wazuh Integration
→ IOC matched (Indicator or Observable)
→ STIX Sighting auto-created in OpenCTI
→ Enriched alert via Wazuh socket
→ Rule 100302 fires in dashboard
→ Telegram notification delivered
→ Full bidirectional threat intelligence loop
```

Total sightings created in lab: 8+

IOC types covered: IPv4, IPv6, Domain, URL, MD5, SHA1, SHA256, Filename, Windows Process

Storage overhead: Zero (no CDB files in production mode)

Detection latency: < 1 second from log event to alert

AegisryxLabs — SOC Automation Series

Wazuh + OpenCTI + Telegram | June 2026